

CANDEX HANDBOOK

Deployment

How Candex ships — the API as a Docker Compose stack per environment, CI/CD, the reverse proxy, and the mobile app's store releases.

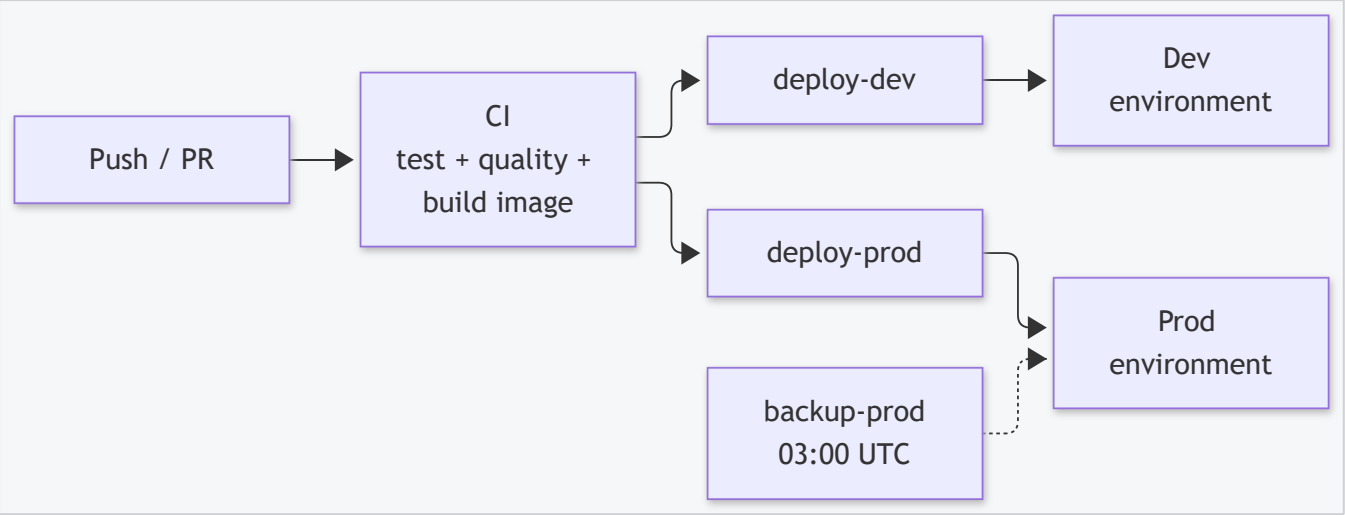
Written by
Eliott Barbet

Version 0.23.0
November 2026
candex.app

Deployment

This is how Candex is built, shipped, and operated. The API runs as a **Docker Compose** stack per environment; the mobile app ships to the stores with **Fastlane**. To run it on your machine, see Local Installation; for the system around it, Architecture & Overview.

The pipeline at a glance



Environments

Each environment is its own Compose stack (an API container + a PostgreSQL 17 container):

Environment	Compose file	Exposure
local	`compose.local.yaml`	Throwaway stack via `scripts/candex.sh` (Local Installation)
dev	`compose.dev.yaml`	`api-dev` + `db-dev`; API on host `8081`, debugger on `5005`
prod	`compose.prod.yaml`	`api-prod` + `db-prod`; **no published ports** — reached through the reverse proxy

Each API container runs with `SPRING_PROFILES_ACTIVE` set to its environment, restarts `unless-stopped`, and keeps rotated `json-file` logs.

The API image

The API is built by a multi-stage `Dockerfile` — compiled with Maven, then run on a lean JRE:

```
FROM maven:3.9.9-eclipse-temurin-21 AS build
# ... mvn -B -DskipTests package
FROM eclipse-temurin:21-jre
COPY --from=build /app/target/*.jar /app/app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Configuration is entirely environment-driven (database, security, SQL console, Uploadcare) — see Local Installation for the variables. Nothing secret is baked into the image.

Operating a stack

`scripts/candex.sh` operates a running dev or prod stack from the server — without stopping it:

```
./scripts/candex.sh psql <dev|prod>        # a psql session
./scripts/candex.sh logs <dev|prod>        # follow the API logs
./scripts/candex.sh dump <dev|prod>        # pg_dump the database
./scripts/candex.sh exec <dev|prod> "<SQL>"
```

CI/CD

GitHub Actions drive integration and delivery (`.github/workflows`):

Workflow	Role
`ci.yml`	Quality gate on push / PR — build, tests, lint (and Qodana)
`deploy-dev.yml` • `deploy-prod.yml`	Deploy the API to dev / prod
`backup-prod.yml`	Scheduled production database backup

The dashboard has its own `ci.yml` quality gate.

Reverse proxy & TLS

> **To confirm**
>
> The production API publishes no ports; the public entrypoint and TLS are handled by a **reverse proxy at the server level** (a `CADDY_EMAIL` variable points to Caddy). The exact proxy configuration lives outside these repositories and is not documented here yet.

Mobile app releases

The app ships to the **Play Store** and **App Store** with **Fastlane**, triggered manually from GitHub Actions. Version bump and changelog stay local and human-reviewed — CI only deploys an existing tag:

```
npm run release 1.2.0            # bump + annotated tag x.y.z (message = changelog)
git push && git push origin 1.2.0
# GitHub ▸ Actions ▸ "Deploy to Stores" ▸ Run workflow (store: both / play / appstore)
```

`deploy.yml` (a manual `workflow_dispatch`) builds the chosen tag and submits it through the Fastlane lanes `android deploy` (AAB → Play production) and `ios deploy` (IPA → App

Store Connect).

Where to go next

- Database backups → Backups
- Run it on your machine → Local Installation
- The API itself → Back-end